

3.Desarrollo de módulos y modelos (II)

SISTEMAS DE GESTIÓN EMPRESARIAL

2ºDAM

A solid orange horizontal bar at the bottom of the slide.

Bibliografía

[Odoo 11 Development Essentials - Third Edition](#)

ISBN-13: 978-1788477796

[Odoo 11 Development Cookbook](#)

ISBN-13: 978-1788471817

[Working with Odoo 11 - Third Edition: Configure, manage, and customize your Odoo system](#)

ISBN-13: 978-1788476959

Índice

- 1. Estructura de un modelo**
2. Tipos de modelos
3. Atributos de un modelo
4. Campos de un modelo
5. Campos relacionales

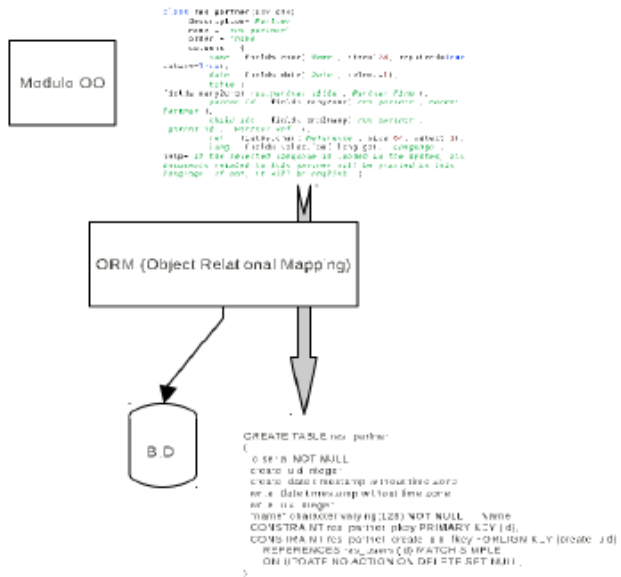
Open Object Framework



- El proyecto Open Object Framework es el germen de OpenERP, antecesor a Odoo.
- Es un marco de desarrollo (framework) de tipo RAD (Rapid Application Development)
- Es un proyecto Open Source

La capa ORM

Traduce los modelos (clases) desarrollados en los módulos Odoo en tablas y relaciones de la base de datos.



Estructura de un modelo

- Los modelos de Odoo representan los objetos de negocio y su clase base siempre será **odoo.models.BaseModel**.
- Cada modelo Odoo se implementa mediante una clase Python con una estructura específica.
- Los modelos de Odoo se crean heredando uno de los siguientes tipos: **AbstractModel**, **Model**, **TransientModel**.

Índice

1. Estructura de un modelo
- 2. Tipos de modelos**
3. Atributos de un modelo
4. Campos de un modelo
5. Campos relacionales

Modelo: AbstractModel

- La clase AbstractModel no es más que un alias de la clase BaseModel.
- Utilizada cuando existe alguna característica que puede ser heredada por un conjunto de modelos regulares (Model).
- Por lo tanto, si se desea probar el modelo abstracto, se debe heredar múltiples modelos de tipo `odoo.models.Model`.

Modelo: Model

- Clase Python para modelos Odoo persistentes en bases de datos regulares.
- Una vez que se crea un modelo regular, el sistema crea la instancia de la clase una vez por base de datos.

```
from odoo import models, fields, api  ← Importaciones de modelos, campos y api

class ClassName(models.Model):        ← Herencia de models.Model
    _name = 'models.ClassName'        ← Nombre interno del modelo

    name = fields.Char()
    value = fields.Integer()
    value2 = fields.Float(compute="_value_pc", store=True)
    description = fields.Text()
    }                                  ← Atributos del modelo

    @api.depends('value')
    def _value_pc(self):                ← Métodos del modelo
        self.value2 = float(self.value) / 100
```

Modelo: TransientModel

- La clase **TransientModel** tiene su propia superclase **odoo.models.TransientModel**.
- Modelo para datos temporales, almacenados en la base de datos pero de vez en cuando se eliminan de forma automática.
- Los permisos de acceso a un modelo transitorio se encuentran restringidos. Todos los usuarios pueden crear nuevos registros en estas tablas temporales, pero sólo pueden acceder a aquellos registros en los que su propietario sea el usuario en línea.
- El superusuario del sistema no tiene restricciones de acceso a cualquier registro de este tipo de modelos.

Tipos de modelos

- Los nombres de los modelos deben ser globalmente únicos. Debido a esto, se suelen conformar anteponiendo el nombre de la aplicación/módulo al nombre del modelo. Por ejemplo, *library.book*
- Se aconseja utilizar CamelCase para nombrar las clases Python de los modelos.
- En la gran mayoría de los modelos de Odoo, las clases se basan en la clase `models.Model`. Estos tipos de modelos tienen persistencia permanente de la base de datos: se crean tablas de base de datos para ellos y sus registros se almacenan hasta que se borran explícitamente.

Índice

1. Estructura de un modelo
2. Tipos de modelos
- 3. Atributos de un modelo**
4. Campos de un modelo
5. Campos relacionales

Atributos de un modelo

- Los atributos más utilizados son:
 - **`_name`** es el identificador interno del modelo Odoo que estamos creando. Obligatorio cuando se crea un nuevo modelo.
 - **`_description`** es un título fácil de usar para los registros del modelo, que se muestra cuando se ve el modelo en la interfaz de usuario. Opcional pero recomendado.
 - **`_order`** establece el orden predeterminado para utilizar cuando se exploran los registros del modelo o se muestran en una vista de lista. Es una cadena de texto que se usará como cláusula SQL order by, por lo que puede ser cualquier cosa que puedas utilizar allí.

Atributos de un modelo

- Para completar, hay un par de más atributos que se pueden utilizar en casos avanzados:
 - ***_rec_name*** indica el campo a utilizar como la descripción del registro cuando se hace referencia desde campos relacionados, tales como una relación de varios a uno. De forma predeterminada, utiliza el campo de nombre, que es un campo común en los modelos. Pero este atributo nos permite usar cualquier otro campo para ese propósito.
 - ***_table*** es el nombre de la tabla de la base de datos que soporta el modelo. Por lo general, se deja que se calcule automáticamente, y es el nombre del modelo con los puntos reemplazados por subrayados. Pero es posible establecer para indicar un nombre de tabla específico.

Atributos de un modelo

- Otros atributos que componen un modelo son:
 - **`_auto`**. Valor booleano asociado a la creación de una tabla en la base de datos.
 - **`_sequence`** es la secuencia SQL para usar en el campo ID.
 - **`_sql_constraints`**. Restricciones SQL.
 - **`_register`** es un valor booleano que indica la visibilidad en el registro ORM.
 - **`_inherit` e `_inherits = {}`** se utilizan para implementar una herencia entre modelos de Python.
 - **`_order`** es un campo de orden predeterminado para los resultados de búsqueda.
 - **`_abstract`** es un valor booleano para confirmar si el modelo es abstracto.
 - **`_transient`** es un valor booleano para confirmar si el modelo es transitorio.

Índice

1. Estructura de un modelo
2. Tipos de modelos
3. Atributos de un modelo
- 4. Campos de un modelo**
5. Campos relacionales

Campos de un modelo

- Los atributos que definen los datos que almacenará el modelo se deben basar en los tipos predeterminados por Odoo.
- Odoo soporta todos los tipos de datos básicos que se esperan, tales como cadenas de texto, enteros, números de punto flotante, booleanos o fechas.

```
·# String fields:
·name = fields.Char('Name', 40)
·desc = fields.Text('Description')
·state = fields.Selection(
·    [ ('draft', 'New'), ('open', 'Started'),
·      ('done', 'Closed') ], 'State')
·docs = fields.Html('Documentation')
·# Numeric fields:
·sequence = fields.Integer('Sequence')
·perc_complete = fields.Float('% Complete', (3, 2))
·# Date fields:
·date_effective = fields.Date('Effective Date')
·date_changed = fields.Datetime('Last Changed')
·# Other fields:
·fold = fields.Boolean('Folded?')
·image = fields.Binary('Image')
```

Campos de un modelo

- En la mayoría de los casos, el primer argumento es el título del campo. Se utiliza como texto predeterminado para las etiquetas de la interfaz de usuario. Es opcional, y si no se proporciona, un título se generará automáticamente a partir del nombre del campo.
- A continuación una breve descripción de los tipos de datos y los argumentos utilizados para especificarlos:
 - **char** representa una entrada de texto en una sola línea. El segundo parámetro, que es opcional, especifica el tamaño máximo de texto. Se recomienda no usarlo a menos que exista un requisito de negocio que lo requiera, como un número de seguro social con una longitud fija.

```
name = fields.Char('Name', 40)
```

Campos de un modelo

- **Text** difiere poco de **Char**, ya que alberga contenido de texto multilínea. Además espera los mismos argumentos.
- **Selection** es una lista de selección desplegable. El primer argumento es la lista de opciones seleccionables y el segundo es el título de la cadena. El elemento de selección es una lista de tuplas ('value', 'Title'), para el valor almacenado en la base de datos y la correspondiente descripción de interfaz de usuario. Cuando se extiende a través de la herencia, el argumento *selection_add* está disponible para añadir nuevos elementos a una lista de selección existente.

```
state = fields.Selection(  
    [ ('draft', 'New'), ('open', 'Started'),  
      ('done', 'Closed') ], 'State')
```

Campos de un modelo

- **Integer** sólo espera un argumento de cadena para el título del campo.
- **Float** tiene un segundo argumento opcional, una tupla (x,y) con la precisión del campo: x es el número total de dígitos; de éstos, y son dígitos decimales.

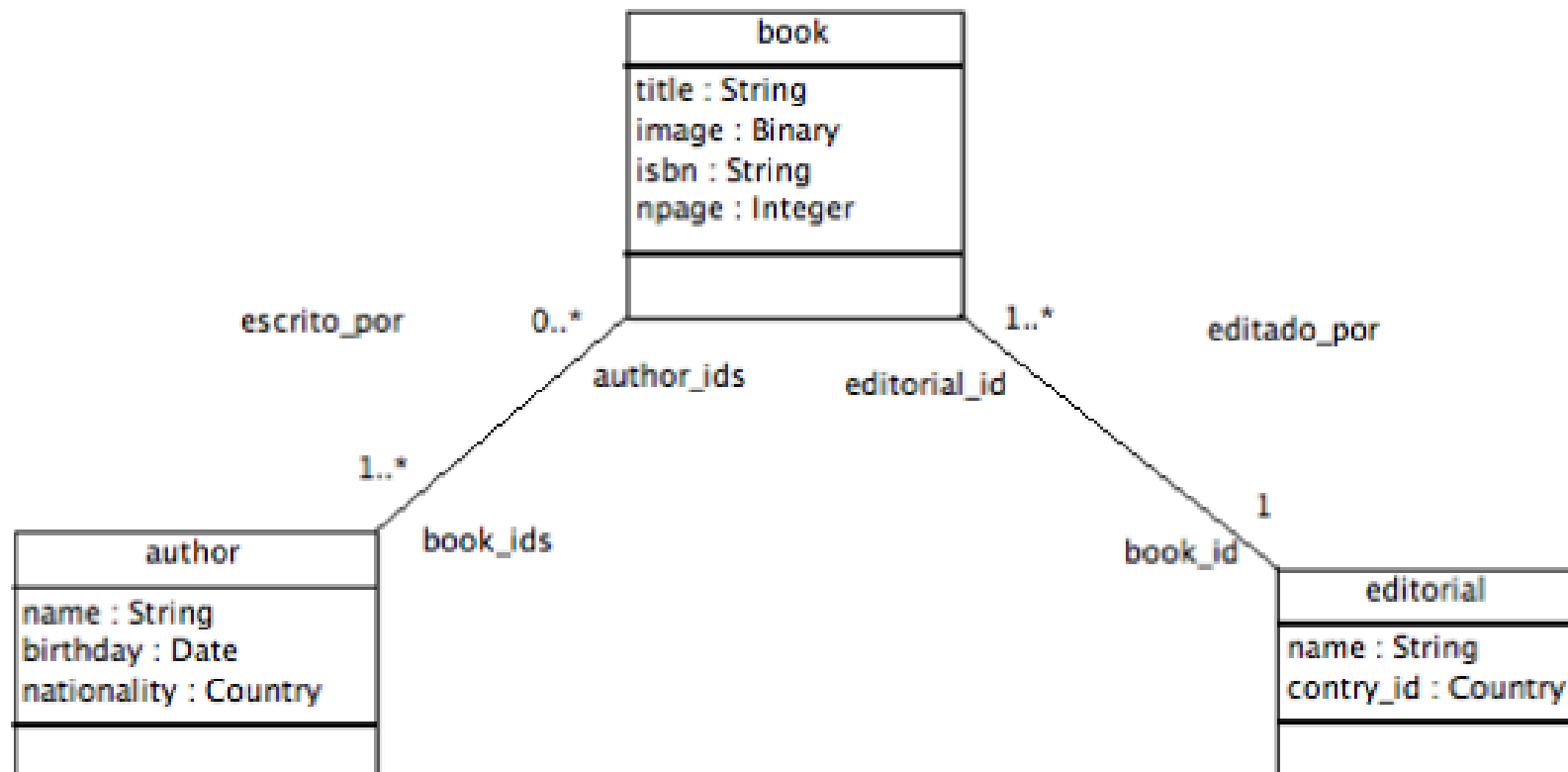
```
perc_complete = fields.Float('% Complete', (3, 2))
```

- Los campos **Date** y **Datetime** sólo esperan el argumento para el título de campo. Por razones históricas, el ORM maneja sus valores en un formato de cadena. Las funciones auxiliares se deben utilizar para convertirlas en objetos de fecha real. También los valores de fecha y hora se almacenan en la base de datos en tiempo UTC pero presentadas en hora local, utilizando las preferencias de zona horaria del usuario.
- **Boolean** tiene valores True o False, como puedes esperar, y sólo espera un argumento de cadena para el título del campo.
- **Binary** almacena datos binarios de tipo archivo y también espera sólo el argumento de cadena. Pueden ser manejados por código Python usando cadenas codificadas en base64.

Campos de un modelo

- Los campos tienen atributos que se pueden establecer al definirlos.
- La forma de hacerlo es mediante el par *nombre de atributo = valor*.
- Una lista exhaustiva de estos atributos sigue a continuación:
 - **default** - establece un valor predeterminado para el campo. Puede ser un valor estático, como un cadena o una referencia invocable, ya sea una función con nombre o una función anónima (una expresión lambda).
 - **translate** se aplica solo a los campos Char, Text y Html, y hace que el contenido del campo se considere traducible.
 - **help** – proporciona el texto para los tooltip de ayuda de un campo
 - **readonly=True** – hace que el campo no sea editable en la interfaz de usuario.
 - **required=True** – hace que el campo sea obligatorio de rellenar en la interfaz de usuario.
 - **index=True** – agrega un índice en la base de datos sobre el campo que se defina, lo que consigue operaciones de búsqueda más rápidas, aunque por contra se ralenticen las operaciones de escritura.

Ejemplo: library



Modelo book.py (v1)

```
from odoo import models, fields, api

class book(models.Model):
    ... _name = 'library.book'

    ... title = fields.Char("Título", size=128)
    ... image = fields.Binary("Imagen")
    ... isbn = fields.Char("ISBN", size=16)
    ... npage = fields.Integer("Nº de páginas")
    ... type = fields.Selection([("fantasia", "Novela fantástica"),
    ...                          ("historia", "Ensayo histórico"),], 1, "Género")
```

Modelo author.py (v1)

```
from odoo import models, fields, api

class author(models.Model):
    .. _name = 'library.author'

    .. name = fields.Char("Nombre", size=64, required=True, readonly=False)
    .. nationality = fields.Many2one("res.country", "Nacionalidad")
    .. birthday = fields.Date("Fecha de nacimiento", size=16)
    ..
```

Modelo editorial.py (v1)

```
from odoo import models, fields, api

class editorial(models.Model):
    ... _name = 'library.editorial'

    ... name = fields.Char("Editorial", size=64, required=True)
    ...
```

Índice

1. Estructura de un modelo
2. Tipos de modelos
3. Atributos de un modelo
4. Campos de un modelo
- 5. Campos relacionales**

Campos relacionales

- Se utilizan para reflejar las relaciones entre los modelos.
- No siempre es necesario o recomendable establecer los campos relacionales en ambos sentidos de la relación
- Hay tres tipos de campos relacionales:
 - **fields.Many2one** * ----- 1 (muchos a uno)
 - **fields.One2many** 1 ----- * (uno a muchos)
 - **fields.Many2many** * ----- * (muchos a muchos)

Campos relacionales:

Many2one (Muchos a uno)

- Los campos **Many2one** (muchos a uno) agregan una columna en la base de datos con el ID del registro relacionado.
- A nivel de la base de datos, también se creará una restricción de clave foránea, asegurando que los ID almacenados sean una referencia válida a un registro en la tabla relacionada.
- Debe considerarse agregar un índice a estos campos relacionales. Esto se puede hacer agregando el atributo *index=True*.

Campos relacionales:

Many2one (Muchos a uno)

```
field_id = fields.Many2one("nombre.otro.modelo",  
    "Etiqueta del campo",  
    parametros_opcionales)
```

- En el modelo book tenemos el campo que relaciona con el modelo editorial
- Muchos libros editados por una editorial → Muchos a uno

```
editorial_id = fields.Many2one("library.editorial", "Editorial")
```

Campos relacionales:

One2many (Uno a muchos)

- Los campos **One2many** (uno a muchos) son lo contrario de las relaciones Many2one (muchos a uno), y aunque se agregan a los modelos al igual que otros campos, no tienen representación real en la base de datos.
- Deben considerarse atajos programáticos que habilitan vistas para representar estas listas de registros relacionados.
- En muchas ocasiones no es interesante agregar esta relación por no ser relevante, por ejemplo Idioma → Persona, País → Producto.

Campos relacionales:

One2many (Uno a muchos)

```
field_ids = fields.One2many("nombre.otro.modelo",  
                             "Campo otro modelo",  
                             "Etiqueta del campo",  
                             parametros_opcionales)
```

- En el modelo editorial podemos definir la relación inversa.
- Para definir esta relación se tiene que haber definido previamente la relación Many2one inversa.
- Una editorial edita muchos libros → Uno a muchos.

```
book_ids = fields.One2many("library.book", "editorial_id", "Libros")
```

Campos relacionales:

Many2many (Muchos a muchos)

- Las relaciones Many2many (muchos a muchos) tampoco agregan columnas en las tablas para los modelos.
- Este tipo de relación se representa en la base de datos utilizando una tabla de relación intermedia, con dos columnas para almacenar las dos ID relacionadas.
- Odoo maneja automáticamente la creación de esta tabla de relaciones. El nombre de la tabla de relaciones es, de forma predeterminada, creado utilizando el nombre de los dos modelos relacionados más un sufijo `_rel`.
- Mediante el atributo `relation` podemos definir explícitamente el nombre de la tabla de la relación.
- Un caso a tener en cuenta es cuando los nombres de las dos tablas son lo suficientemente grandes para que los identificadores de la base de datos generados automáticamente superen el límite de PostgreSQL de 63 caracteres.

Campos relacionales:

Many2many (Muchos a muchos)

```
field_ids = fields.Many2many("nombre.otro.modelo",  
                             string="Etiqueta campo",  
                             parametros_opcionales)
```

- Un autor puede escribir muchos libros y un libro puede ser escrito por varios autores

author.py

```
libros_ids = fields.Many2many("library.book",  
                              string="Libros",  
                              # relation='library_book_library_partner_rel' # opcional  
autores_ids = fields.Many2many("library.author",  
                                string="Autores",  
                                # relation='library_book_library_partner_rel' # opcional  
                                )
```

book.py

Campos relacionales:

Campos de referencia

- En los campos relacionales normales (Many2many, One2Many y Many2One) la relación entre dos modelos es fija.
- Los campos de referencia soportan establecer relaciones dinámicas, de forma que el mismo campo puede referenciar a más de un modelo.

```
refers_to = fields.Reference(  
    [('res.user', 'User'), ('res.partner', 'Partner')],  
    'Refers to')
```

Campos relacionales:

Campos de referencia

- La definición del campo es similar a un campo Selection, pero aquí la lista de selección contiene los modelos que se pueden usar.
- En la interfaz de usuario, el usuario primero elegirá un modelo de la lista disponible y luego elegirá un registro de ese modelo.
- Esto puede llevarse a otro nivel de flexibilidad ya que existe una tabla de modelos referenciables, donde deberemos incluir aquellos modelos que queramos que sean referenciables.
- Está disponible en la Ajustes → Estructura de la base de datos → Modelos referenciables

Campos relacionales:

Campos computados

- Podemos tener campos cuyos valores sean calculados automáticamente por una función, en lugar de simplemente leyendo el valor almacenado de una base de datos.
- Un campo calculado se declara como un campo normal, pero tiene un argumento para definir la función que se utiliza para su cálculo.
- En la mayoría de los casos, los campos calculados implican escribir algo de lógica empresarial y el uso de la API ORM, de forma que su estudio queda pospuesto hasta avanzar más en la asignatura.

Campos relacionales:

Campos computados

```
· # Campo calculado: el nombre a mostrar estará formado  
· # por el nombre y los apellidos concatenados  
· # store=True --> Se calcula cada vez que se modifica  
· # un campo que intervenga en el cálculo  
· display_name = fields.Char(compute=compute_display_name, store=True)  
  
· # Especificación de campos que intervienen en el cálculo  
· @api.depends('name', 'surname')  
· def compute_display_name(self):  
·     · # Este método se invocará cada vez  
·     · # que el campo name o surname se modifique  
·     · pass  
·  
· # Campo calculado: el nombre a mostrar estará formado  
· # por el nombre y los apellidos concatenados  
· # store=False --> Se calcula cada vez que se visualice  
· display_name = fields.Char(compute=compute_display_name_v2, store=False)  
·  
· def compute_display_name_v2(self):  
·     · # Este método se invocará cada vez que el campo se visualice  
·     · pass
```

Ejercicio completo: book, autor, editorial

```
from odoo import models, fields, api

class book(models.Model):
    .. _name = 'library.book'

    .. title = fields.Char("Título", size=128)
    .. image = fields.Binary("Imagen")
    .. isbn = fields.Char("ISBN", size=16)
    .. npage = fields.Integer("Nº de páginas")
    .. type = fields.Selection([("fantasia", "Novela fantástica"),
    .. .. ("historia", "Ensayo histórico"), ], "Género")

    .. editorial_id = fields.Many2one("library.editorial", "Editorial")
    ..
    .. autores_ids = fields.Many2many("library.author",
    .. .. string="Autores",
    .. .. # relation='library_book_library_partner_rel' # opcional
    .. .. )
```

Ejercicio completo: book, autor, editorial

```
from odoo import models, fields, api

class author(models.Model):
    .. _name = 'library.author'

    .. name = fields.Char("Nombre", size=64, required=True, readonly=False)
    .. nationality = fields.Many2one("res.country", "Nacionalidad")
    .. birthday = fields.Date("Fecha de nacimiento", size=16)
    ..
    .. libros_ids = fields.Many2many("library.book",
    ..         string="Libros",
    ..         # relation='library_book_library_partner_rel' # opcional
    ..         )
```

Ejercicio completo: book, autor, editorial

```
from odoo import models, fields, api

class editorial(models.Model):
    .. _name = 'library.editorial'
    ....
    .. name = fields.Char("Editorial", size=64, required=True)
    .. book_ids = fields.One2many("library.book", "editorial_id", "Libros")
    ....
```